Programming Project 2 - Sets and Multiset Operations

Name: Connor
Course: CS 2430
Semester: Summer 2026

This project implements ordinary sets and multisets. Sets just track if something is there or not, multisets go further and track how many times it appears. The dataset is a fruit store inventory where three stores each carry some mix of ten fruits and the operations answer things like which fruits two stores share or how much combined stock they hold. Ordinary sets use boolean arrays where each index is a fruit in the universal set, true means the store has it, and this maps directly to a bit string. Multisets use integer arrays over the same universal set where each index holds a count instead of a boolean, union takes the max, intersection takes the min, difference subtracts but does not go below zero, and sum just adds. Both parts are split into separate files under the Sets:: and Multisets:: namespaces. Three test cases were run for both parts, Test Case 1 uses Store A and Store B which only share Mango, Test Case 2 uses Store A and Store C which share Apple and Lemon, and Test Case 3 uses Store B and Store C as an edge case with minimal overlap. All three stores have at least two elements with counts greater than one and the difference operation was checked to confirm it never goes negative. If you send in a bool array the compiler picks the set version of a function and if you send in an int array it picks the multiset version so overloading works fine, though if you store everything in an int array you lose that type difference. Figuring out which type you need from a natural language query is pretty hard since you have to parse intent from word choice and users do not always phrase things clearly, and going from multiset to set is fine since you just drop the counts but going the other way gives you counts of only 0 or 1 which does not gain you anything. The practical answer is to either ask for clarification or default to multiset since it is more general and can be downgraded, and if you add more types like fuzzy sets on top of that the ambiguity just compounds. The implementation was straightforward once the data structures were picked and the discussion questions were the more interesting part, especially thinking through how much ambiguity shows up in plain English queries.

Results

are in a txt file in src/ called `output.txt`